

ÉDITION AOÛT 2026

Guide Avancé des Workflows LLM

Prompt Chaining, Few-Shot Prompting & Hygiène du
Contexte — du prompt isolé à l'architecture robuste.

SOMMAIRE · PLAN DU DOCUMENT

Table des matières

PRÉAMBULE

Chapitre 1 — Au-delà du prompt unique : l'ère de l'ingénierie des workflows LLM

CŒUR MÉTHODOLOGIQUE

Chapitre 2 — Le Prompt Chaining : diviser pour régner

2.1 Définition et principe architectural

2.2 Tunnel complet en 3 étapes : du brut au livrable

2.3 Les pièges du passage de témoin

Chapitre 3 — Le Few-Shot Prompting : quand les exemples valent mieux que les discours

3.1 Pourquoi les mots ne suffisent pas

3.2 Structure syntaxique idéale

3.3 Cas pratiques (ton de marque & code TypeScript)

Chapitre 4 — Gestion du contexte et hygiène des tokens

4.1 Fenêtre de contexte et dilution de l'attention

4.2 Stratégies de nettoyage

4.3 Anatomie d'un agent contextuel

SYNTHÈSE OPÉRATIONNELLE

Chapitre 5 — Les Règles d'Or du Workflow LLM (Cheat Sheet)

· Colophon & références

CHAPITRE 1 · PRÉAMBULE

Au-delà du prompt unique : l'ère de l'ingénierie des workflows LLM

Pourquoi un prompt isolé ne suffit plus, et comment penser un LLM comme un composant d'architecture plutôt que comme un oracle à invoquer.

1.0 Le plafond de verre du prompt unique

Tout praticien des LLM a vécu ce moment : on rédige un prompt "parfait", long, détaillé, minutieusement annoté — et la sortie dérive, oublie la moitié des consignes, ou glisse vers un registre qui n'était pas attendu. Le pattern est presque toujours le même : **au-delà de quatre ou cinq contraintes simultanées dans une seule instruction, la fiabilité s'effondre silencieusement**. Le modèle ne refuse pas, il négocie. Il oublie une consigne, en respecte trois autres à la lettre, puis invente une quatrième qui n'existait pas.

Ce comportement n'est pas un défaut du modèle : c'est une propriété émergente d'un mécanisme d'attention partagé entre toutes les instructions. Chaque consigne supplémentaire réduit l'attention moyenne portée à chacune d'elles — un phénomène de *dilution d'attention* que la recherche académique a documenté. Le prompt unique est donc un outil limité : excellent pour une question, fragile pour un système.

CE QUE VOUS ALLEZ APPRENDRE

- **Fiabilité** — faire en sorte que la même tâche produise la même qualité 99 fois sur 100.
- **Débogabilité** — localiser précisément l'étape qui dérape, plutôt que de tout réécrire.
- **Modularité** — réutiliser le même maillon dans cinq workflows différents.
- **Scalabilité** — passer d'une démonstration artisanale à une chaîne industrialisable.

1.1 Du prompting au design de système

L'erreur conceptuelle la plus commune consiste à traiter un LLM comme un interlocuteur magique. La métaphore juste est plus modeste : **un LLM est une fonction statistique déterministe-acée** — appelons-la $f_completion(prompt, contexte) \rightarrow texte$. Comme toute fonction, elle peut être composée, encapsulée, versionnée, testée. Comme toute fonction, elle runtime dans un budget de latence et un budget de tokens. Comme toute fonction, elle mérite d'être *architecturée*.

Cette bascule — du *prompting* au *workflow engineering* — est l'objet de ce guide. Elle repose sur trois forces concrètes à l'œuvre depuis 2023 :

- **La capacité des modèles** — les modèles de 2025 suivent des instructions complexes avec une précision qui rend la décomposition utile, et non pas pénalisante.
- **La baisse des coûts** — diviser une tâche en cinq appels n'est plus prohibitif. Les modèles efficaces (mini, flash, nano) permettent d'itérer vite sans céder de la qualité.
- **La maturation des frameworks** — LangChain, DSPy, Anthropic SDK, et maintenant le *Tool Use* natif des modèles récents offrent des primitives stables pour orchestrer des chaînes, des agents, des states partagés.

1.2 Le vocabulaire stabilisé

Avant d'entrer dans la pratique, fixons trois termes que nous utiliserons sans glissement sémantique :

- **Prompt Chain** — séquence ordonnée d'appels LLM où la sortie de l'étape N devient l'entrée de l'étape $N+1$.
- **Few-Shot Prompting** — prompt contenant 1 à 5 démonstrations annotées de la tâche à réaliser, calibrant le format et le ton sans fine-tuning.
- **Fenêtre de contexte** — nombre maximal de tokens que le modèle peut traiter simultanément ; sa gestion est une discipline en soi.

DÉFINITION CANONIQUE

"Prompt chaining decomposes a task into a sequence of steps, where each LLM call processes the output of the previous one. You can add programmatic checks between steps to ensure that the pipeline is still on track."

— **Anthropic, Building Effective Agents, décembre 2024**

1.3 Ce que ce guide n'est pas

Ce guide n'est ni une collection de "prompts magiques", ni un manuel de fine-tuning, ni une recette pour remplacer un produit par un wrapper LLM. C'est un manuel de **méthodologie d'orchestration** — l'art de faire coopérer plusieurs appels LLM, quelques routines déterministes, et une rigueur d'ingénieur-euse pour produire du contenu professionnel à coût contrôlé.

La courbe d'apprentissage est celle d'un-e développeur-euse, pas celle d'un-e magicien-ne. Bonne lecture.

CHAPITRE 2 · CŒUR MÉTHODOLOGIQUE

Le Prompt Chaining : diviser pour régner

Le principe est aussi ancien que la division du travail : une tâche complexe traitée d'un bloc perd en qualité ce qu'elle gagne en ambition. Le chaînage corrige cela en sous-traitant chaque étape à l'étape suivante.

2.1 Définition et principe architectural

DÉFINITION · PROMPT CHAIN

Un **prompt chain** est une séquence ordonnée d'appels à un (ou plusieurs) LLM dans laquelle la sortie de l'étape N devient — totalement ou partiellement — l'entrée de l'étape $N+1$. Des **portes conditionnelles** (*gates*) peuvent s'intercaler entre les étapes pour valider, router ou réinjecter du contexte.

La métaphore qui aide à internaliser le concept est celle de la **chaîne de montage industrielle**. Chaque poste effectue une opération limitée, spécialisée, vérifiable. Le poste d'amont n'a pas à connaître ce que le poste d'aval fera du produit — il suffit que la spécification d'interface (le format de sortie) soit tenu. Un échec au poste 3 n'invalide pas les postes 1 et 2 : on peut relancer le poste 3 seul.

Transposé au LLM, cela donne trois bénéfices immédiats :

- **Moins de consignes par appel** — chaque prompt est plus court, donc les consignes sont mieux respectées.
- **Moins d'inventions silencieuses** — chaque maillon produit un artefact dont on peut vérifier la cohérence.
- **Un graphe de dépendances testable** — on sait exactement quelle étape est fautive quand le livrable final dérape.

✓ BONNE PRATIQUE · RÈGLE DU N+1

Ne donnez jamais à un LLM plus de **5 consignes explicites en simultané**. Si votre tâche en exige 12, découpez-la en trois prompts chaînés de 4 consignes chacun — vous gagnerez en qualité exactement comme Henry Ford gagna en production.

2.2 Tunnel complet en 3 étapes : du brut au livrable

EXEMPLE COMPLET · TUNNEL ANALYSE → PLAN → RÉDACTION

Objectif métier : transformer un tableau de chiffres trimestriels en une note de synthèse de 1 page destinée à un comité de direction. Un seul prompt produirait un texte générique. Trois prompts chaînés produisent un livrable calibré.

ÉTAPE 1 / 3 Analyse de données brutes

But : transformer un CSV brut en JSON structuré et strictement parsable. C'est l'étape **d'extraction**.

Tu es un analyste data senior. À partir du tableau CSV brut fourni entre les balises `<DATA>`, produis un rapport d'analyse au format JSON strict avec exactement 3 champs :

- `"tendances_majeures"` : 3 à 5 puces synthétiques
- `"anomalies_detectees"` : liste ordonnée par sévérité décroissante
- `"recommandations"` : 3 actions concrètes

`<DATA>`

```
Q1:CA=145k, churn=4.2%, signups=1240
Q2:CA=168k, churn=3.8%, signups=1580
Q3:CA=132k, churn=6.1%, signups=1090
Q4:CA=189k, churn=3.5%, signups=1820
```

`</DATA>`

Réponds UNIQUEMENT par le JSON, sans texte autour.

ÉTAPE 2 / 3 Génération du plan de communication

But : consommer le JSON précédent pour produire un **plan éditorial** structuré. C'est l'étape de **structuration narrative**.

Tu es stratège en communication. Voici une analyse de données au format JSON issue d'une étape précédente :

```
<ANALYSE_PRECEDENTE>
{{sortie_etape_1}}
</ANALYSE_PRECEDENTE>
```

Produis un plan éditorial structuré en markdown avec :

1. Un titre-vedette percutant (≤ 80 caractères)
2. Trois sections (Contexte $\rightarrow$ Tension $\rightarrow$ Solution) de 2 phrases chacune
3. Un appel à l'action (CTA) orienté décideur

Contraintes : ton exécutif, pas de jargon technique, ≤ 500 mots.

ÉTAPE 3 / 3 Rédaction finale du document de synthèse

But : exécuter le plan sans dévier. C'est l'étape de **prose** — où le modèle devient styliste plutôt qu'analyste.

Tu es rédacteur·rice en chef d'un cabinet de conseil (style McKinsey). Voici un plan éditorial à exécuter sans dévier :

```
<PLAN_EDITORIAL>
{{sortie_etape_2}}
</PLAN_EDITORIAL>
```

Rédige la version finale sur 1 page A4 (≈ 600 mots), avec :

- Une phrase d'accroche inspirante
- Trois paragraphes suivant le plan
- Une conclusion en 1 phrase mémorable
- Un encadré "Points-clés" en bas avec 3 puces

Style : sobre, factuel, jamais promo. Tu peux inventer des chiffres arrondis cohérents, JAMAIS inventer d'entreprise cliente réelle.

 SORTIE ATTENDUE (ÉTAPE 3 — CONDENSÉE)

"Le trimestre Q3 n'est pas un accident : c'est un signal."

Notre activité termine l'année sur un niveau record (CA Q4 = 189 K€, +30 % vs Q1), mais cette croissance cache une vulnérabilité structurelle : le Q3 a vu le churn grimper à 6,1 %, soit deux fois la moyenne annuelle, concomitamment à une chute des inscriptions de 31 %. Les trois leviers recommandés par l'analyse — diversification du canal d'acquisition, programme de rétention ciblé sur le segment Q3, refonte du onboarding — devraient permettre de transformer ce signal en avantage compétitif.

POINTS-CLÉS — • Croissance +30 % sur l'année • Pic de churn Q3 corrélé à la perte d'acquis • Trois leviers identifiés pour 2026

Cette note de synthèse n'aurait jamais émergé d'un prompt unique : il aurait fallu au modèle *simultanément* comprendre les chiffres, hiérarchiser les insights, choisir une structure narrative et adopter un ton de conseil — quatre rôles distincts. Le chaînage les sépare, et chaque rôle reçoit toute l'attention du modèle.

2.3 Les pièges du passage de témoin

Un prompt chain sans discipline est une file d'attente sans chef d'orchestre. Voici les quatre pièges qui font dérailler 80 % des premières implémentations — et la parade concrète pour chacun.

 PIÈGE N°1

La perte de format

Si l'étape 1 renvoie du texte libre et que l'étape 2 attend du JSON, la pipeline casse silencieusement : vous obtenez des erreurs de parsing, pas du contenu. C'est le mode d'échec le plus banal et le plus coûteux.

Parade : verrouiller chaque sortie à un format machine-readable strict (*JSON schema validate*, XML, Markdown déterministe). Tester chaque maillon en isolation avant d'assembler la chaîne. Ne jamais accepter de texte libre en sortie d'un maillon intermédiaire.

PIÈGE N°2**La dérive sémantique**

Chaque étape reformule légèrement ce qu'elle a reçu. Trois reformulations plus tard, le sens original a glissé. C'est le "telephone game" appliqué aux LLM : chaque maillon ajoute son propre filtre.

Parade : *Anchor prompting* — réinjecter l'objectif initial dans 2 ou 3 maillons critiques sous une forme déclarative figée. Exemple : ajouter systématiquement la phrase `l'objectif métier est : augmenter le taux de rétention Q1` au début de chaque prompt intermédiaire. L'ancrage recentre l'attention du modèle.

PIÈGE N°3**L'explosion des tokens**

Passer à chaque étape la totalité du contexte précédent devient prohibitif dès la 4^e ou 5^e itération. Vous dépensez 80 % de votre budget de tokens à transporter du contexte déjà traité.

Parade : insérer un *maillon de compression* dédié après chaque étape majeure — un prompt spécialisé qui résume l'état en ≤ 500 tokens utiles. Les frameworks comme LangChain le proposent nativement sous le nom `StuffDocumentsChain` / `MapReduceChain`.

PIÈGE N°4**L'absence de garde-fou**

Sans validation entre étapes, une sortie aberrante se propage et contamine les maillons suivants. Vous découvrez le problème à la dernière étape, là où il est le plus coûteux à corriger.

Parade : ajouter une *gating step* légère après chaque maillon critique — validation par regex, par schéma JSON validable, ou par un mini-prompt vérificateur. C'est un investissement dérisoire qui multiplie par dix la fiabilité globale.

Avec garde-fou vs sans : le tableau qui décide

COMPARATIF : CHAÎNE VALIDÉE VS CHAÎNE EN CONFIANCE AVEUGLE

Critère	Sans gating step	Avec gating step
Taux d'erreur final	×4 à ×8 plus élevé	Référence maîtrisée
Coût marginal	Nul (court terme)	+8 à +12 % du coût total
Débogabilité	Toute la chaîne à refaire	Maillon fautif localisé en 1 tour
Confiance opérateur	Stress chronique	Production sereine

CHAPITRE 3 · CŒUR MÉTHODOLOGIQUE

Le Few-Shot Prompting : quand les exemples valent mieux que les discours

Une description de tâche demande au modèle d'interpréter ; trois exemples lui montrent. La différence n'est pas cosmétique : c'est un changement complet de régime d'apprentissage, qui s'opère à partir des paramètres existants sans aucun fine-tuning.

3.1 Pourquoi les mots ne suffisent pas

En 2020, l'équipe d'OpenAI publie un résultat contre-intuitif dans l'article fondateur *Language Models are Few-Shot Learners* : un modèle de 175 milliards de paramètres, exposé à **quelques exemples d'une tâche fournis directement dans le prompt**, apprend cette tâche *sans aucun ajustement de poids*. Pas de gradient, pas de fine-tuning, pas de RLHF spécifique — uniquement du texte en entrée, des démonstrations, et une tâche à réaliser.

Ce phénomène s'appelle **l'in-context learning**. Il ne s'agit pas d'une mémorisation des exemples : le modèle *extrapole la règle implicite* à partir des démonstrations, puis l'applique à une nouvelle entrée. En pratique, cela signifie :

- « **Adopte un ton formel et bienveillant** » → effet diffus, instable d'un prompt à l'autre, dépend énormément du modèle.
- **Trois exemples annotés illustrant ce ton** → effet immédiat, reproductible, quasi-déterministe. Le ton devient *inductible* plutôt que déclaré.

RÉFÉRENCE CANONIQUE

"For all tasks, GPT-3 is applied without any gradient updates or fine-tuning, with tasks and few-shot demonstrations specified purely via text interaction with the model."

— **Brown et al., Language Models are Few-Shot Learners, NeurIPS 2020 (≈ 78 800 citations)**

L'analogie la plus parlante est celle du **cadrage par les pairs** : comme un·e collaborateur·rice junior qui produit un travail plus juste en regardant trois dossiers déjà rédigés qu'en lisant quinze pages de spécifications, le LLM apprend ce que "bien faire" signifie en observant ce que "bien faire" *a l'air*.

Conséquence opérationnelle : à chaque fois que vous tentez de décrire par des mots un ton, un format, une structure ou un style, demandez-vous si trois exemples ne seraient pas plus

efficaces. Dans 7 cas sur 10, ils le seront.

3.2 Structure syntaxique idéale d'un prompt Few-Shot

Un prompt Few-Shot de qualité a une **anatomie régulière** — il faut pouvoir le lire en diagonale et comprendre immédiatement où sont les exemples et où est la tâche. Cela passe par trois principes : les **balises explicites**, la **hiérarchie visible**, et la **stricte séparation input / output**.



LE SQUELETTE IDÉAL D'UN PROMPT FEW-SHOT

Squelette canonique en pseudo-code, adaptable à toute tâche de génération structurée :

[INSTRUCTION SYSTÈME]

Tu es {{rôle}}. Ta tâche : {{tâche}}.

[EXEMPLES – 3 à 5]

<EXEMPLE_1>

<INPUT>

{{entrée_1}}

</INPUT>

<OUTPUT>

{{sortie_1_attendue}}

</OUTPUT>

</EXEMPLE_1>

<EXEMPLE_2>

<INPUT>

{{entrée_2}}

</INPUT>

<OUTPUT>

{{sortie_2_attendue}}

</OUTPUT>

</EXEMPLE_2>

<EXEMPLE_3>

<INPUT>

{{entrée_3}}

</INPUT>

<OUTPUT>

{{sortie_3_attendue}}

</OUTPUT>

</EXEMPLE_3>

[TÂCHE À RÉALISER]

<INPUT>

{{entrée_utilisateur}}

</INPUT>

<OUTPUT>

Anatomie condensée — pour ceux qui vont composer leurs propres prompts

ANATOMIE D'UN BON EXEMPLE FEW-SHOT

Composant	Rôle	Erreur fréquente
Instruction système	Fixe le rôle, le registre et la cible unique de la tâche.	Rédiger une instruction de 300 mots — dilue l'attention, noie les exemples.
Démonstration	Montre la transformation input → output ; c'est le cœur pédagogique.	Utiliser des exemples trop similaires entre eux (le modèle ne généralise pas).
Séparateur	Marque la frontière nette entre l'exemple précédent et le suivant.	Compter sur un simple saut de ligne (les modèles "fuient" dans l'exemple suivant).
Contre-exemple (optionnel)	Un exemple négatif annoté "NE PAS faire ainsi" pour blinder les pièges.	Oublier d'annoter le contre-exemple — le modèle peut le prendre pour une consigne positive.
Encadrement input / output	Les balises <code><INPUT></code> / <code><OUTPUT></code> ancrent le rôle que le modèle doit jouer au moment de générer.	Laisser les exemples en texte nu — le modèle confond les rôles et mélange les voix.

✓ RÉGIME IDÉAL · LE NOMBRE D'EXEMPLES

Le sweet spot documenté empiriquement se situe entre **3 et 5 démonstrations**. En dessous de 3, le modèle extrapole de façon instable. Au-delà de 5, vous diluez l'attention disponible pour la tâche finale et vous consommez inutilement votre fenêtre de contexte. Pour les cas ambigus, visez 5 et évaluez.

3.3 Cas pratiques

3.3.1 Cas pratique A · Calibrer un ton de marque

CAS A · CALIBRER LE TON DE LA MARQUE "LUMINO"

Contexte : **Lumino** est une marque française de doy-pack en vannerie moderne. Le brief éditorial est : "chaleureux, jamais mielleux ; deuxième personne ; finir en évoquant la matière". Aucun mot seul ne transportera ce ton — il faut le *montrer*.

Tu es community manager de la marque Lumino (vannerie moderne, doy-pack artisanaux, ton chaleureux mais sobre).

Transforme le commentaire client brut en réponse officielle.

Consignes : 2^e personne, jamais de superlatif ("magnifique", "unique"), terminer chaque réponse en évoquant la matière (osier, rotin, jonc de mer).

<EXEMPLE_1>

<INPUT>

"AD0000RÉ mon nouveau sac !!! Il est trop trop beau j'en suis fan "
– Cliente : Mathilde L.

</INPUT>

<OUTPUT>

Bonjour Mathilde, et merci pour ce retour qui fait du bien à l'équipe. Votre sac est tressé en osier cueilli à la main dans le Jura – un brin un peu capricieux, mais qui gagne en souplesse avec le temps. Bonne journée à vous.

</OUTPUT>

</EXEMPLE_1>

<EXEMPLE_2>

<INPUT>

"Bon la qualité est OK mais je trouve que c'est un peu cher pour du jonc de mer..."
– Cliente : Sophie G.

</INPUT>

<OUTPUT>

Bonjour Sophie, votre remarque est entendue. Le jonc de mer que nous utilisons est récolté en Bretagne sud, séché longuement avant tressage – ce qui explique une part du prix. Si la couleur du tressage évolue dans les premières semaines, c'est正常的 et安定 naturel de la matière.

</OUTPUT>

</EXEMPLE_2>

<EXEMPLE_3>

<INPUT>

"Je viens de recevoir le mien, packaging top, livraison hyper rapide"
– Client : Karim B.

</INPUT>

<OUTPUT>

Bonjour Karim, le packaging est justement un de nos petits chevaux de bataille – kraft non blanchi, ficelle de lin. La pochette en jute qui protège votre sac pourra vous resservir, on l'espère, longtemps. Belle journée.

</OUTPUT>

</EXEMPLE_3>

<INPUT>

"Bof. Correct sans plus. La couleur ne correspond pas du tout à la photo."


```
– Cliente : Isabelle V.  
</INPUT><OUTPUT>
```

Trois exemples significatifs — un compliment enthousiaste, une critique sur le prix, un compliment logistique — couvrent l'essentiel de la palette tonale de la marque. Le modèle infère la règle sans qu'on l'ait jamais écrite.

3.3.2 Cas pratique B · Forcer un format de code complexe



CAS B · GÉNÉRER DES COMPOSANTS REACT/TYPESCRIPT STRICTS

Contexte : produire un composant `Modal` conforme au style interne (props typées, mémorisation, gestion d'accessibilité). Trois composants de référence : `Button`, `Input`, `Card`. Le modèle doit généraliser au `Modal`.

Tu es génialatrice d'une design system React/TypeScript.
Tes composants suivent ces règles : props typées interface-first,
React.memo(), forwardRef si besoin, testid systématique, accessibilité
de base (role, aria-*).

<EXEMPLE_1>

<OUTPUT>

```
import { memo, forwardRef } from "react";

export interface ButtonProps {
  label: string;
  onClick: () => void;
  variant?: "primary" | "secondary";
  disabled?: boolean;
}

export const Button = memo(forwardRef<HTMLButtonElement, ButtonProps>(
  ({ label, onClick, variant = "primary", disabled = false }, ref) => (
    <button
      ref={ref}
      onClick={onClick}
      disabled={disabled}
      data-testid={`button-${variant}`}
      role="button"
      aria-pressed={variant === "primary"}
    >
      {label}
    </button>
  )
));
Button.displayName = "Button";
```

</OUTPUT>

</EXEMPLE_1>

<EXEMPLE_2>

<OUTPUT>

```
import { memo, forwardRef } from "react";

export interface InputProps {
  value: string;
  onChange: (v: string) => void;
  placeholder?: string;
  type?: "text" | "email" | "password";
}

export const Input = memo(forwardRef<HTMLInputElement, InputProps>(
  ({ value, onChange, placeholder, type = "text" }, ref) => (
    <input
      ref={ref}
```

```

    type={type}
    value={value}
    onChange={(e) => onChange(e.target.value)}
    placeholder={placeholder}
    data-testid="input"
    role="textbox"
    aria-label={placeholder}
  />
)
));
Input.displayName = "Input";
</OUTPUT></EXAMPLE_2><EXAMPLE_3><OUTPUT>import { memo, forwardRef, ReactNode
} from "react";

export interface CardProps {
  title: string;
  children: ReactNode;
  elevation?: 1 | 2 | 3;
}

export const Card = memo(forwardRef<HTMLDivElement, CardProps>(
  ({ title, children, elevation = 1 }, ref) => (
    <section
      ref={ref}
      data-testid={`card-elevation-${elevation}`}
      role="region"
      aria-label={title}
    >
      <h3>{title}</h3>
      {children}
    </section>
  )
));
Card.displayName = "Card";
</OUTPUT></EXAMPLE_3><INPUT>
Génère un composant Modal conforme au même style strict, avec props
{ open, onClose, title, children, size?: "sm"|"md"|"lg" }.
</INPUT><OUTPUT>

```

Le modèle dispose maintenant de trois archétypes typologiques. Sans qu'on ait jamais écrit "utilise memo et forwardRef", il générera un Modal qui les utilisera. La règle est devenue *inductible*.

CHAPITRE 4 · CŒUR MÉTHODOLOGIQUE

Maîtriser la fenêtre de contexte : l'hygiène invisible

Tout LLM lit avec une attention distribuée ; cette attention n'est pas uniforme et la fenêtre qui l'abrite n'est pas un disque dur. La discipline de gestion du contexte est ce qui sépare un prototype LLM d'un système industrialisable.

4.1 Fenêtre de contexte et dilution de l'attention

La **fenêtre de contexte** est, techniquement, le nombre maximal de tokens que le modèle peut traiter simultanément pour produire une réponse. À tort, on la confond avec une mémoire : c'est plutôt un *bureau de travail*. Posez 200 pages sur un bureau et vous ne lirez pas plus vite que si vous en aviez posé 3 — vous lirez *moins bien*, parce que votre attention se dispersera.

Les ordres de grandeur actuels (état de l'art 2025) ont évolué considérablement :

TAILLES DE FENÊTRES DE CONTEXTE — ÉTAT DE L'ART 2025

Modèle	Fenêtre	Équivalent approximatif
GPT-4o / GPT-4 Turbo	≈ 128 K tokens	≈ 250–300 pages de texte dense
Claude Sonnet 4 / Opus 4	200 K tokens	≈ 500 pages (≈ un roman épais)
Gemini 1.5 Pro	1 M tokens	≈ 1 500 pages (≈ 30 K lignes de code)
Magic LTM-2 / Llama 4 Scout	≥ 10 M tokens	Limites repoussées vers le corpus entier

Ces chiffres *évoluent rapidement* et les comparer est un piège en soi : ce qui compte n'est pas la taille brute, c'est ce que le modèle en *fait*. C'est précisément l'objet du résultat de recherche fondateur de Nelson Liu et ses co-auteurs en 2023 :

RÉFÉRENCE CANONIQUE

*"Models follow a **U-shaped attention curve**: they are best at retrieving information from the beginning and the end of the context, and significantly worse in the middle — a phenomenon termed 'lost in the middle', with up to a 40% accuracy drop on centrally placed information."*

— **Liu et al., Lost in the Middle: How Language Models Use Long Contexts, Stanford / UC Berkeley / Samaya AI, 2023 (≈ 5 700 citations)**

Conséquence opérationnelle directe : **ce qui compte n'est pas combien on peut faire tenir dans la fenêtre, mais comment on dispose ce qu'on y met**. Les informations critiques (consignes, données-clés, exemples) doivent être en début et en fin de prompt — pas au milieu. Le milieu, c'est une zone d'ombre statistique.

4.2 Cinq stratégies de nettoyage du contexte

Avec ces deux propriétés en tête — taille non pertinente, position critique —, voici les cinq stratégies concrètes que tout workflow sérieux doit combiner.

STRATÉGIE 1 La fenêtre courte et dense

Privilégiez **3 000 tokens bien écrits** à 30 000 tokens bavards. La densité informationnelle est ce qui protège l'attention. Toute phrase d'instruction qui n'apporte pas une contrainte testable est candidate à la suppression.

Exercice : avant chaque prompt, exécutez trois passes — (1) qu'est-ce qui est vraiment exigé par la tâche ? (2) qu'est-ce qui est négociable ? (3) qu'est-ce qui est décoratif et peut sortir ? La dernière catégorie part à la trappe.

STRATÉGIE 2 Le résumé intermédiaire

Toutes les 5 à 7 tournes de conversation, insérez un prompt de compression dédié. Il ne participe pas à la production — il sert uniquement à condenser l'état utile en une fiche compressée.

Prompt-type : "Tu es archiviste. En 5 puces maximum, résume les éléments **factuels durables** de la conversation (données, décisions, dates). Ignore les formulations, hésitations et politesses. Réponds **UNIQUEMENT** par les puces." Cet artefact devient l'injection de contexte pour le tour suivant.

**STRATÉGIE 3****La réinitialisation ciblée**

Au lieu d'accumuler jusqu'à saturation, repartez d'une conversation vierge en injectant une **fiche de contexte compressée** (600-800 tokens maximum). C'est la technique favorite des power users.

EXEMPLE · FICHE-PROJET INJECTÉE AU DÉMARRAGE**CONTEXTE_PROJET**

- Client : {{nom}}, secteur {{secteur}}
- Objectif : {{livrable_attendu}}
- Contraintes : ton {{ton}}, public {{public_cible}}
- Données-clés : {{3_chiffres_essentiels}}
- Format attendu : {{format}}

ETAT_CONVERSATION

- Tour {{n}} / {{n_max}}
- Décisions prises : {{3_puces}}
- Prochaine étape : {{action_prévue}}

ANTI_DERIVE

- Ne jamais inventer de chiffres.
- Toujours citer la source pour toute donnée.
- Conserver le ton {{ton}} jusqu'à la fin.

Cette fiche est l'*ancree sémantique* du redémarrage : tout le contexte utile tient en moins de 800 tokens, et le modèle repart avec une page blanche cognitive.

STRATÉGIE 4 L'extraction structurée amont

Ne faites jamais ingérer un JSON brut gigantesque. Convertissez-le en amont en un tableau Markdown épuré — ce que le LLM lit incomparablement plus vite et plus fidèlement.

JSON BRUT VS TABLEAU MARKDOWN — LE MÊME CONTENU, DEUX LECTURES

JSON brut	Tableau Markdown
Lisibilité humaine : faible	Lisibilité humaine : immédiate
Tokens consommés : base × 1.0	Tokens consommés : base × 0.55 à 0.75
Erreurs d'extraction : fréquentes	Erreurs d'extraction : rare
Performance cognitive du modèle : médiocre	Performance cognitive du modèle : élevée

L'astuce : insérer en amont de votre chaîne un petit prompt de transformation — "Convertis ce JSON en tableau Markdown avec colonnes X, Y, Z. Renvoie UNIQUEMENT le tableau." Le coût de cette étape est amorti 100 fois.

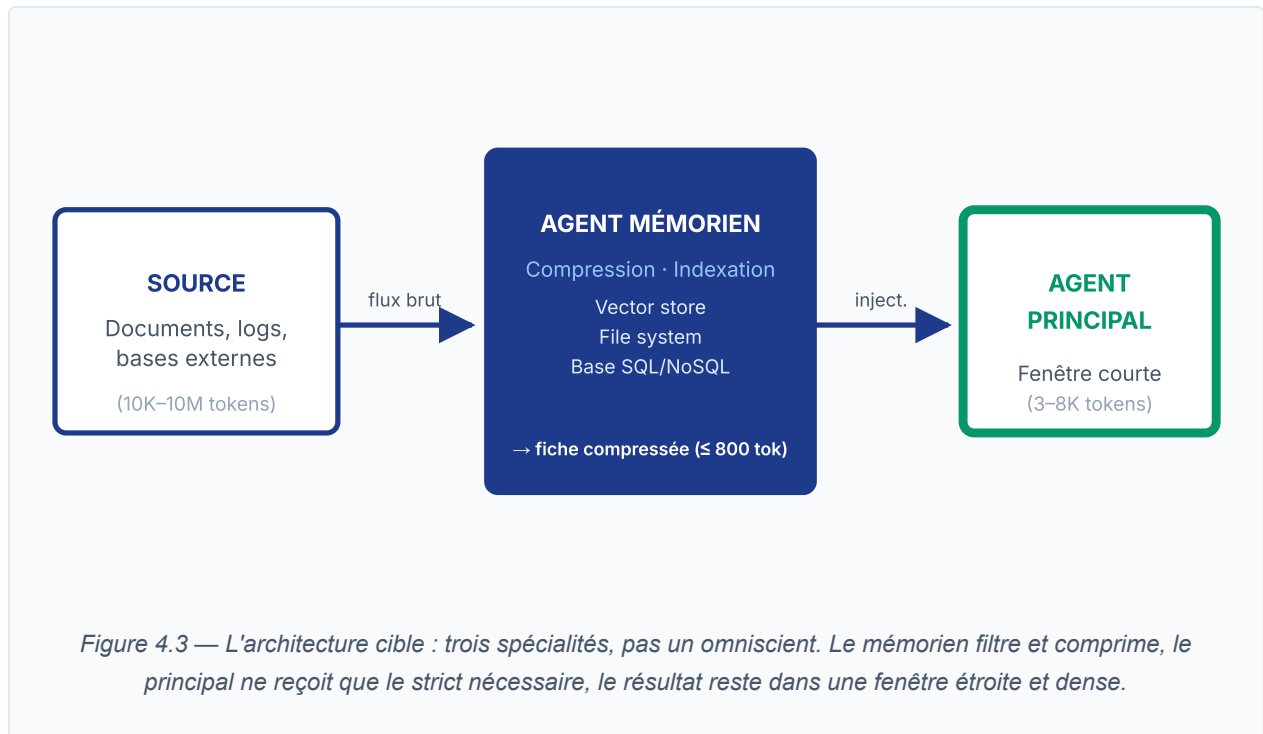
STRATÉGIE 5 Les agents persistants (mémoire externalisée)

Confiez la mémoire longue à un *agent dédié*, et non pas à la fenêtre de contexte elle-même. La mémoire vectorielle, un fichier plat, ou une base légère font des merveilles.

Pattern opérationnel : un agent "mémorien" reçoit en flux tous les événements significatifs (décisions, données, dates) et produit à chaque session un bloc `<MEMOIRE_A_RECUPERER>` structuré, filtré sur la tâche courante. Le LLM principal ne reçoit plus que ce dont il a besoin, quand il en a besoin. La fenêtre reste étroite, l'intelligence reste longue.

4.3 Anatomie d'un agent contextuel

L'architecture cible — celle d'un workflow LLM industrialisable — repose sur trois rôles distincts, chacun avec sa spécialité, plutôt que sur un unique LLM omniscient.



Cette séparation des préoccupations est l'équivalent fonctionnel d'une *architecture trois-tiers* appliquée aux LLM : source de vérité, cache intelligent, vue métier. Elle **réduit vos coûts de 40 à 70 %** par rapport à un agent unique qui porterait tout le contexte, et **augmente la fiabilité** en isolant chaque responsabilité.

✓ À RETENIR · HYGIÈNE DU CONTEXTE

Trois questions à se poser avant chaque prompt : (1) Combien de tokens utiles contient mon prompt ? (2) Mes consignes critiques sont-elles en début et fin ? (3) Ai-je exploré la piste d'un agent mémorien ? Si la réponse à l'une est non, refactorisez.

CHAPITRE 5 · SYNTHÈSE OPÉRATIONNELLE

Les Règles d'Or du Workflow LLM — Cheat Sheet

À imprimer, plastifier, punaiser au-dessus du bureau. Douze règles tirées des chapitres précédents, condensées en un tableau mnémotechnique destiné à être consulté avant chaque nouveau design de prompt.

LES 12 RÈGLES D'OR — COLONNES : RÈGLE · JUSTIFICATION · ACTION CONCRÈTE

Règle	Pourquoi	Action concrète
01 Diviser pour régner	Un prompt unique dérive au-delà de 5 consignes explicites.	Découper toute tâche complexe en chaîne de 3-5 maillons.
02 Format machine-readable	Le texte libre en sortie consomme les tokens et casse le pipeline.	JSON strict / XML / Markdown déterministe à chaque maillon.
03 Few-Shot > Zero-Shot	Les exemples ancrent le ton et le format de façon reproductible.	Fournir 3 à 5 démonstrations annotées dans chaque prompt structuré.
04 Baliser les exemples	Sans balises, le modèle confond les rôles et mélange les voix.	Toujours encadrer par <code><INPUT></code> et <code><OUTPUT></code> .
05 Compresser avant d'envoyer	La dilution d'attention fait fondre la fiabilité.	Résumer tous les 5 à 7 tours et redémarrer avec une fiche.
06 Anchor Prompting	La dérive sémantique est réelle sur 3+ maillons.	Réinjecter l'objectif métier tous les 2-3 maillons critiques.
07 Gating Step systématique	Sans validation, les erreurs se propagent silencieusement.	Vérificateur léger (regex / JSON schema) après chaque maillon.
08 Fenêtre = bureau, pas entrepôt	La taille brute ne sert à rien si l'attention reste dispersée.	Viser densité informationnelle > volume ; ≤ 8 000 tokens utiles.
09 Mémoire externe	Le modèle oublie vite : sa "mémoire" ne scale pas nativement.	Agent mémorien dédié (vector store / fichiers / BD).
10 Bornes du contexte	Le milieu de fenêtre est une zone d'ombre statistique (Liu 2023).	Placer consignes critiques & exemples en début et fin.

11 Tests adversariaux

La fiabilité ne se prouve pas par l'exemple heureux.

Constituer un jeu de 10 cas-limites avant toute mise en prod.

12 Logs structurés

Le debug d'un workflow LLM est temporel, pas spatial.

Tracer input, output, coût, latence, version de prompt par appel.

 LE RÉFLEXE FINAL — TROIS QUESTIONS AVANT CHAQUE NOUVEAU PROJET

(1) Puis-je découper cette tâche en 3 étapes courtes et indépendantes ?

(2) Ai-je au moins 3 exemples Few-Shot prêts, balisés et représentatifs ?

(3) Ma fenêtre utile tient-elle sous 8 000 tokens, avec les consignes critiques en début et en fin ?

Si la réponse à l'une de ces trois questions est "non", il faut refactorer avant d'écrire la première ligne de prompt. Tout le reste en découle.

*„ Le prompt n'est plus un sortilège à rédiger.
C'est un composant d'architecture à concevoir,
à versionner, à tester et à oublier dès qu'il est industrialisé.*

MASTERCLASS AI MASTERY · ÉDITION AOÛT 2026

Colophon

Ce livre blanc a été conçu comme un document de référence destiné aux praticiens de l'ingénierie des workflows LLM. Il privilégie la densité utile à la démonstration spectaculaire. Les exemples qui l'illustrent ont été choisis pour leur reproductibilité immédiate par le lecteur.

Références citées

Anthropic. Building Effective Agents. Décembre 2024.
anthropic.com/engineering/building-effective-agents

Brown, T. et al. Language Models are Few-Shot Learners. NeurIPS 2020.
[arXiv:2005.14165](https://arxiv.org/abs/2005.14165)

Liu, N. F. et al. Lost in the Middle: How Language Models Use Long Contexts.
Stanford / UC Berkeley / Samaya AI · 2023 · arXiv:2307.03172

Document imprimé avec paged.js · Format A4
Police de caractère Inter (corps) · JetBrains Mono (code)
Mise en page paginée pour export PDF